

Corso di Ingegneria del Software Deliverable di progetto	2022-2023
---	-----------

“Ingegneria del Software” 2022-2023

Docente: Prof. Angelo Furfaro

Ken ken new

Data	17/07/2023
Documento	Documento Finale – D3

Team Members		
Nome e Cognome	Matricola	E-mail address
Marco Ziccardi	220889	zccmrc00b08i874l@studenti.unical.it

Sommario

Non è stata trovata alcuna voce d'indice.

1	
---	--

List of Challenging/Risky Requirements or Tasks

Challenging Task	Date the task is identified	Date the challenge is resolved	Explanation on how the challenge has been managed
Creazione griglia di gioco.	04/07/2023	05/07/2023	Per gestire tale funzionalità, si è fatto uso del framework “JavaFX” in quanto semplifica la creazione di interfacce. Mi ha permesso di far interagire l’utente con: una griglia di ‘Buttom’ che una volta premuti identificano i-esimo gruppo, e con una ‘JTextField’ (per far inserire il vincolo numerico) e ‘MenuItem’ (per scegliere una tra le 4 operazioni) per l’inserimento dei vincoli al gruppo. Una volta inseriti tutti i possibili gruppi, alla pressione di un ‘Buttom’ viene caricata una griglia di ‘JTextField’ suddivisa in gruppi per colori in cui l’utente può giocare interagendo.
Importazione ed esportazione griglia da file.	05/07/2023	06/07/2023	Per gestire tali funzionalità si è fatto uso dei pattern Visitor e Builder.
Implementazione dell’algoritmo risolutivo del KenKen.	06/07/2023	06/07/2023	Per gestire tale funzionalità si è fatto uso del pattern Template Method con concretizzazione per il KenKen.
Rappresentazione delle “n” soluzioni iterabili successiva e precedente.	07/07/2023	07/07/2023	Si sono salvate le ‘n’ soluzioni all’interno di una lista iterabile. È data la responsabilità a due oggetti grafici di muoversi nello spazio dei risultati permettendo di stampare la soluzione nelle rispettive TextField.
Implementazione gioca utente e verifica soluzione.	08/07/2023	08/07/2023	Si sono utilizzati degli algoritmi per far comunicare l’interfaccia grafica (che tiene conto dei valori della i-esima celletta) con il risolutore. Il risolutore, lavorando sui dei valori fornitigli in input, può stabilire se i valori rispettano o meno i vincoli e fornisce in output una risposta circa la correttezza o meno dei valori inseriti.

A. Stato dell'Arte

Il KenKen è un gioco di logica e matematica che richiede agli utenti di risolvere puzzle di dimensioni variabili.

Vi sono diversi applicativi software (desktop, mobile e web) che implementano il gioco del kenken.

Tra di loro la variazione in termini di funzionalità offerte è minimale.

Analizzandole si è visto che alcune di queste applicazioni offrono funzionalità avanzate come la generazione casuale dei puzzle, la possibilità di risolvere automaticamente i puzzle e l'integrazione con social media.

Esistono diverse tecniche e algoritmi per la risoluzione del kenken, inclusi il backtracking, la ricerca locale e l'ottimizzazione. Questi algoritmi possono essere utilizzati per calcolare le soluzioni del gioco o per fornire suggerimenti agli utenti durante la fase di gioco.

Alcuni algoritmi possono essere ottimizzati per gestire puzzle di dimensioni più grandi o complessi, riducendo il tempo di calcolo e migliorando le prestazioni complessive del software.

Si è visto che ogni software, in particolare, possiede un'implementazione diversa per la gestione dei repository. Diversi utilizzano un database o un file system per archiviare le configurazioni di gioco.

Diversi software permettono di salvare lo stato della partita e ripristinarla in un secondo momento.

È essenziale sottolineare che lo stato dell'arte è soggetto a rapida evoluzione, con il costante sviluppo di nuove applicazioni e tecnologie. Questa panoramica offre una visione generale dei principali servizi offerti da un software di gioco per il KenKen.

B. Raffinamento dei Requisiti

L'applicativo KenKen è stato sviluppato per fornire agli utenti un ambiente facilitato in grado di supportare l'utente nell'attività di gioco. Inoltre, permette di consultare le soluzioni che la configurazione del gioco ammette. La seguente relazione fornirà una descrizione dettagliata dei servizi offerti dall'applicativo KenKen e dei requisiti non funzionali associati.

Requisiti funzionali:

1. *Creazione dei puzzle*: il software permette di creare una griglia personalizzata potendo selezionare gli 'n' gruppi e su ognuno di esso i vincoli matematici. Molto importante è fornire un riscontro all'utente: "la configurazione scelta non presenta soluzioni" in quanto non avrebbe senso giocarvi.
2. *Importazione/esportazione della griglia su da un file*: il software permette di consultare un file in un formato .xml appositamente definito per leggere e importare una griglia da file. Allo stesso modo, usando lo stesso formato .xml, è possibile esportare la griglia potendola così in un secondo momento rigiocarci. Nel caso dell'importazione viene fornito un riscontro sulla presenza o meno di soluzioni.
3. *Visualizzazione e iterazione delle soluzioni*: il software è in grado di risolvere automaticamente i puzzle. L'utente ha la possibilità di esprimere il numero di soluzioni che desidera consultare. Viene data la possibilità di muoversi nello spazio delle soluzioni possibili.
4. *Supporto per diverse dimensioni di puzzle*: il software supporta puzzle KenKen di dimensioni variabili, griglie di dimensione 3x3 fino a 9x9. Collegandosi con il primo punto, gli utenti possono selezionare la dimensione del puzzle in base alle proprie preferenze e al livello di difficoltà desiderato. Questo servizio assieme al primo punto offre flessibilità e adattabilità per soddisfare le diverse esigenze degli utenti.
5. *Verifica della soluzione dell'utente*: il software permette di verificare se la soluzione temporaneamente inserita è corretta rispetto i vincoli. L'algoritmo di verifica non evidenzia le celle che presentano un contenuto errato, questo per evitare di fornire una esperienza di gioco fortemente agevole e facilitata. L'algoritmo mostra in output la scritta "non tutti i valori sono corretti" poi sta all'utente individuare le celle errate. Si tende a sollecitare un coinvolgimento dell'utente maggiore.
6. *Funzionalità di annullamento dei gruppi in fase di configurazione*: il software dà la possibilità, in fase di configurazione, di annullare l'i-esimo gruppo specificato o resettare addirittura l'intera configurazione della griglia fino a quel punto fornita.

Requisiti non funzionali:

1. Il software deve fornire tempi di risposta rapidi per la risoluzione automatica e per la verifica di una soluzione. Il tempo richiesto per risolvere un puzzle deve essere proporzionale alla sua dimensione e complessità, garantendo un'esperienza all'utente fluida.
2. Il software deve essere provvisto di una interfaccia intuitiva e facile da usare, consentendo agli utenti di interagire senza difficoltà. I messaggi di feedback, di errori e di istruzioni devono essere chiari.
3. Il software deve fornire una soluzione accurata e corretta per tutti i tipi di puzzle KenKen supportati.
4. Il software deve comportarsi in maniera accettabile anche con situazioni non previste
5. Il software deve poter essere in grado di evolvere con nuove funzionalità e/o correzioni.

A.1 Servizi (con prioritizzazione)

Creazione dei puzzle

Descrizione: fornire agli utenti la possibilità di creare a loro piacimento la configurazione dei puzzle con i dovuti vincoli.

Soluzione concettuale: utilizzo di componenti grafici che permettono all'utente una esperienza d'uso elementare. I componenti, interagendo nel loro funzionamento permettono di creare gli 'n' gruppi della griglia con i vincoli.

ID del servizio: S1

Importanza: alta, complessità: alta

Importazione/esportazione della griglia su da un file

Descrizione: fornire agli utenti la possibilità di importare una configurazione di gioco dall'esterno (da file) e di poter esportare una configurazione appena creata.

Soluzione concettuale: definizione di un formato xml con specifici tag che vanno a rappresentare in maniera testuale una configurazione di gioco. In fase di importazione si analizza il file .xml e si creano i corrispettivi oggetti java, per la esportazione si fa il duale della importazione.

ID del servizio: S2

Importanza: media, complessità: media

Visualizzazione e iterazione delle soluzioni

Descrizione: fornire agli utenti la possibilità di consultare l'insieme di soluzione che la configurazione di gioco ammette

Soluzione concettuale: utilizzo di due componenti grafici che permettono all'utente di interagire con un sistema iterativo nel quale sono salvate le n soluzioni individuate dal risolutore automatico.

ID del servizio: S3

Importanza: alta, complessità: bassa

Supporto per diverse dimensioni di puzzle

Descrizione: fornire agli utenti la possibilità di generare puzzle di dimensioni variabili per rispondere a difficoltà di gioco diverse.

Soluzione concettuale: utilizzo di un componente grafico che permette l'inserimento della dimensione della griglia.

ID del servizio: S4

Importanza: alta, complessità: bassa

Verifica della soluzione dell'utente

Descrizione: fornire agli utenti la possibilità di controllare se i valori inseriti rispettano, fino a quel punto, i vincoli, o se i valori inseriti risolvono il puzzle.

Soluzione concettuale: utilizzo di un componente grafico che ad ogni interazione con esso invoca una chiamata verso il risolutore e verifica che i valori numerici forniti sono assegnabili rispetto ai vincoli del puzzle.

ID del servizio: S5

Importanza: alta, complessità: media

Funzionalità di annullamento o reset dei gruppi in fase di configurazione

Descrizione: fornire agli utenti la possibilità di annullare l'ultima configurazione del gruppo e poterla ridefinire o addirittura di cancellare tutte le configurazioni fornite.

Soluzione concettuale: utilizzo di due componenti grafici la cui interazione consente al primo di effettuare l'azzeramento del gruppo appena definito e di avere la possibilità di ridefinire esso e i rispettivi vincoli, al secondo di resettare (cancellare) tutte le configurazioni di gruppo fino a quel punto.

ID del servizio: S6

Importanza: bassa, complessità: bassa

Priorità dell'importanza:

1. S1
2. S3
3. S4
4. S5
5. S2
6. S6

Priorità dell'importanza:

1. S1
2. S2
3. S5
4. S3
5. S4
6. S6

A.2 Requisiti non Funzionali

I requisiti non funzionali più importante nel KenKen:

Usabilità, aspetto fondamentale per garantire un'esperienza di gioco piacevole e senza difficoltà.

1. L'interfaccia utente del gioco del KenKen deve essere intuitiva e facile da comprendere. I pulsanti e i comandi devono essere chiaramente identificabili e posizionati in modo coerente.
2. Durante il gioco, è essenziale fornire un riscontro agli utenti, ad esempio, quando l'utente effettua una mossa, il gioco deve indicare se la mossa è valida o se ci sono errori e anche riguardo la risoluzione corretta di un puzzle.
3. Permettere all'utente di poter personalizzare le preferenze di gioco permettendo di adattare l'esperienza di gioco alle proprie preferenze. Ciò include ad esempio la scelta del livello di difficoltà della griglia.

Gli aspetti chiave relativi all'usabilità possono essere diversi e molteplici. Non si limitano a quelli di cui sopra.

Prestazioni, il gioco deve essere in grado di gestire il caricamento rapido dei puzzle, la risposta istantanea alle mosse degli utenti e un gameplay fluido senza interruzioni.

Manutenibilità, il gioco deve essere facilmente manutenibile per consentire manutenzione di tipo: correttiva (correzione di bug), adattiva (adattare il sistema a cambiamenti del contesto) e perfettiva (estensioni delle funzionalità). Raggiungibile attraverso una progettazione modulare, codice ben documentato e pratiche di sviluppo software che favoriscono la manutenibilità.

Affidabilità, il gioco deve essere stabile e affidabile, evitando crash o comportamenti imprevisti. Richiede un'attenta gestione degli errori, test.

Robustezza, il gioco deve in grado di fornire comportamenti accettabili anche in circostanze non previste come ad esempio "file non trovato" per il ripristino della griglia.

Portabilità, il gioco deve poter essere eseguito su diverse piattaforme hardware e software.

A.3 Scenari d'uso dettagliati

Scenario: Scelta configurazione del gioco:

1. L'utente avvia l'applicazione e accede all'interfaccia utente;
2. Può scegliere se specificare la configurazione del gioco o caricarla da file system.

Scenario: Configurazione del gioco manuale:

1. L'utente avvia l'applicazione e accede all'interfaccia utente;
2. Sceglie la configurazione manuale;
3. L'utente interagisce con una serie di componenti grafici che permettono di specificare e salvare l'i-esimo gruppo;
4. L'utente conferma di dover caricare la griglia.

Scenario: Configurazione del gioco da file:

1. L'utente avvia l'applicazione e accede all'interfaccia utente;
2. Sceglie la configurazione da file;
3. Inserisce il percorso del file dove caricare;
4. La griglia viene caricata automaticamente.

Scenario: Giocare e verificare i vincoli:

1. L'utente carica un'istanza di gioco tra quelle salvata o crea una nuova configurazione;
2. L'utente inserisce i numeri nelle caselle del KenKen;
3. L'utente in qualunque momento può verificare la correttezza.
4. L'applicazione fornisce un riscontro immediato all'utente sul soddisfacimento dei vincoli, stampando a video l'esito della verifica.

Scenario: Calcolo e visualizzazione delle soluzioni:

1. L'utente richiede al sistema di calcolare le soluzioni della configurazione del gioco utilizzando la tecnica del backtracking;
2. L'applicazione calcola le soluzioni e le mostra all'utente;
3. L'utente può navigare sullo spazio delle soluzioni tramite dei pulsanti "Successiva" e "Precedente". Consente di confrontare e analizzare le varie alternative.

Scenario: Esportazione configurazione di gioco

1. L'utente carica un'istanza di gioco tra quelle salvata o crea una nuova configurazione;
2. L'utente esporta la configurazione di gioco.

Questi scenari coprono le principali funzionalità dell'applicazione KenKen, dalla specifica della configurazione del gioco alla gestione del file system, al gioco stesso e al calcolo delle soluzioni.

A.5 Assunzioni

Per soddisfare il seguente requisito: “l’utente può abilitare il controllo del soddisfacimento dei vincoli man mano che si inseriscono i numeri” si è deciso di implementare un ‘Button’ che, una volta cliccato, mostra un messaggio sulla ‘TextField’. Il messaggio indica se i valori inseriti rispettano o meno i vincoli. Si è deciso di non evidenziare le celle che contengono valori numerici, poiché si vuole coinvolgere maggiormente l’utente e limitare la facilità del gioco.

Per rappresentare e quindi salvare su file system una specifica configurazione di gioco, si è deciso di utilizzare un formato .xml. Il file XML descrive una struttura per rappresentare una griglia con gruppi di punti e i vincoli associati a ciascun gruppo. Di seguito è spiegato il significato di ciascun tag:

Tag **<griglia>**: è il tag radice del documento XML e l'elemento principale della griglia. Contiene un elemento **<dimensione>** e uno o più elementi **<gruppo>** che descrivono i gruppi di punti e i vincoli associati.

Tag **<dimensione>**: il testo (numero) racchiuso all’interno rappresenta il numero che specifica la dimensione della griglia.

Tag **<gruppo>**: contiene uno o più elementi **<punto>**. Il testo racchiuso tra i tag "gruppo" rappresenta il valore e l'operazione associata al gruppo.

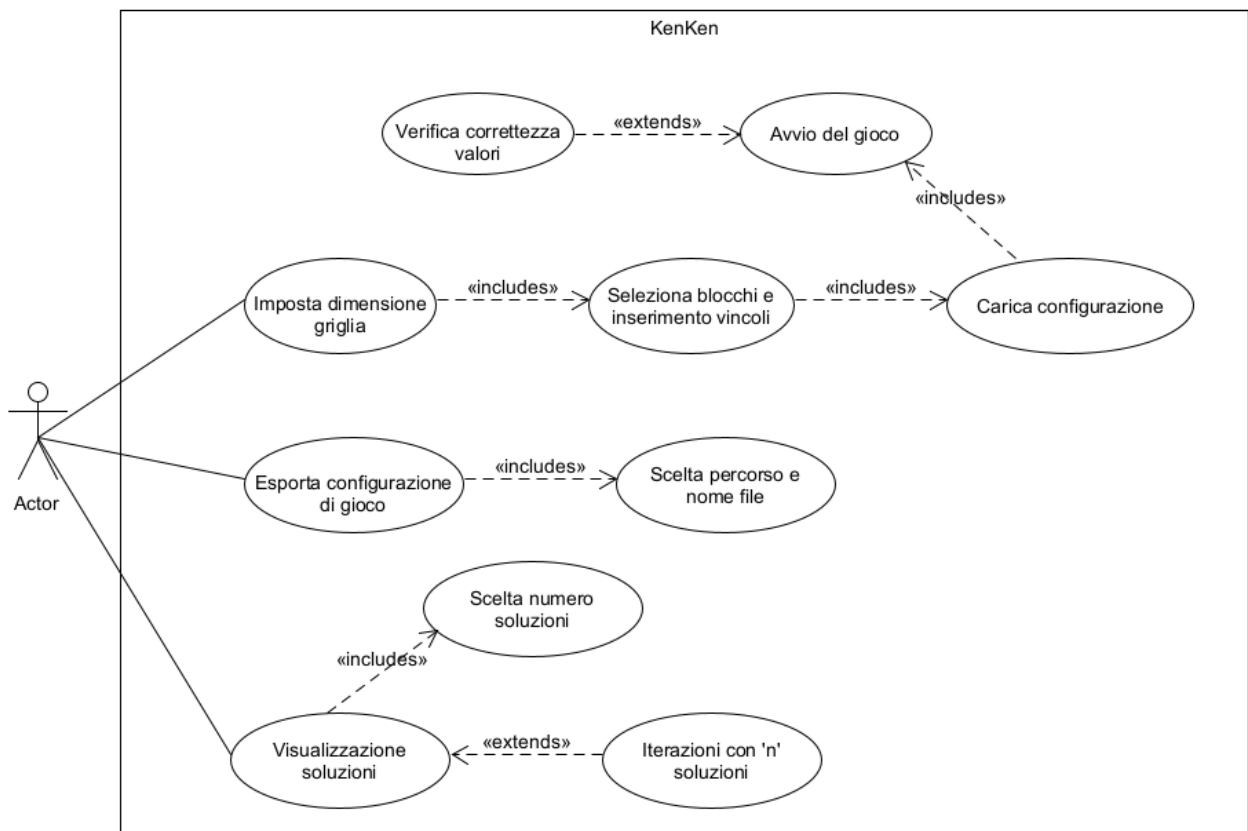
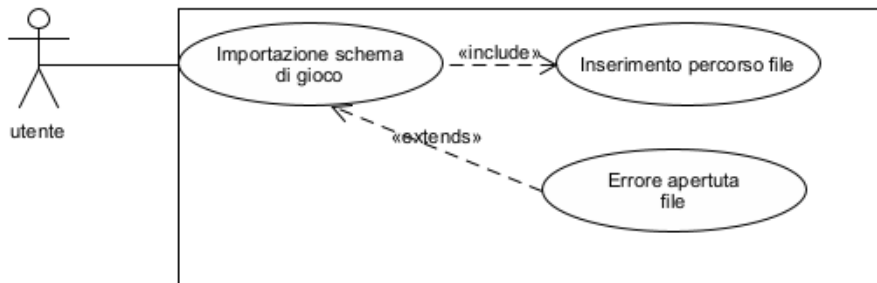
Tag **<punto>**: rappresenta un punto all'interno del gruppo. Ogni elemento "punto" contiene un elemento **<riga>** e un elemento **<colonna>**. Quest’ultimi sono tag foglia. Il testo racchiuso tra il tag "riga" e il tag "colonna" rappresenta il numero della riga e della colonna del punto.

Di seguito è riportato un esempio generale di file .xml

```
<griglia>
  <dimensione> "numero" </dimensione>
  <gruppo>
    "valore operazione"
    <punto>
      <riga> "numero" </riga>
      <colonna> "numero" </colonna>
    </punto>
    <punto>
      <riga> "numero" </riga>
```

```
        <colonna> "numero" </colonna>
    </punto>
</gruppo>
<gruppo>
    "valore operazione"
    <punto>
        <riga> "numero" </riga>
        <colonna> "numero" </colonna>
    </punto>
    <punto>
        <riga> "numero" </riga>
        <colonna> "numero" </colonna>
    </punto>
</gruppo>
</griglia>
```

A.6 Use Case Diagrams

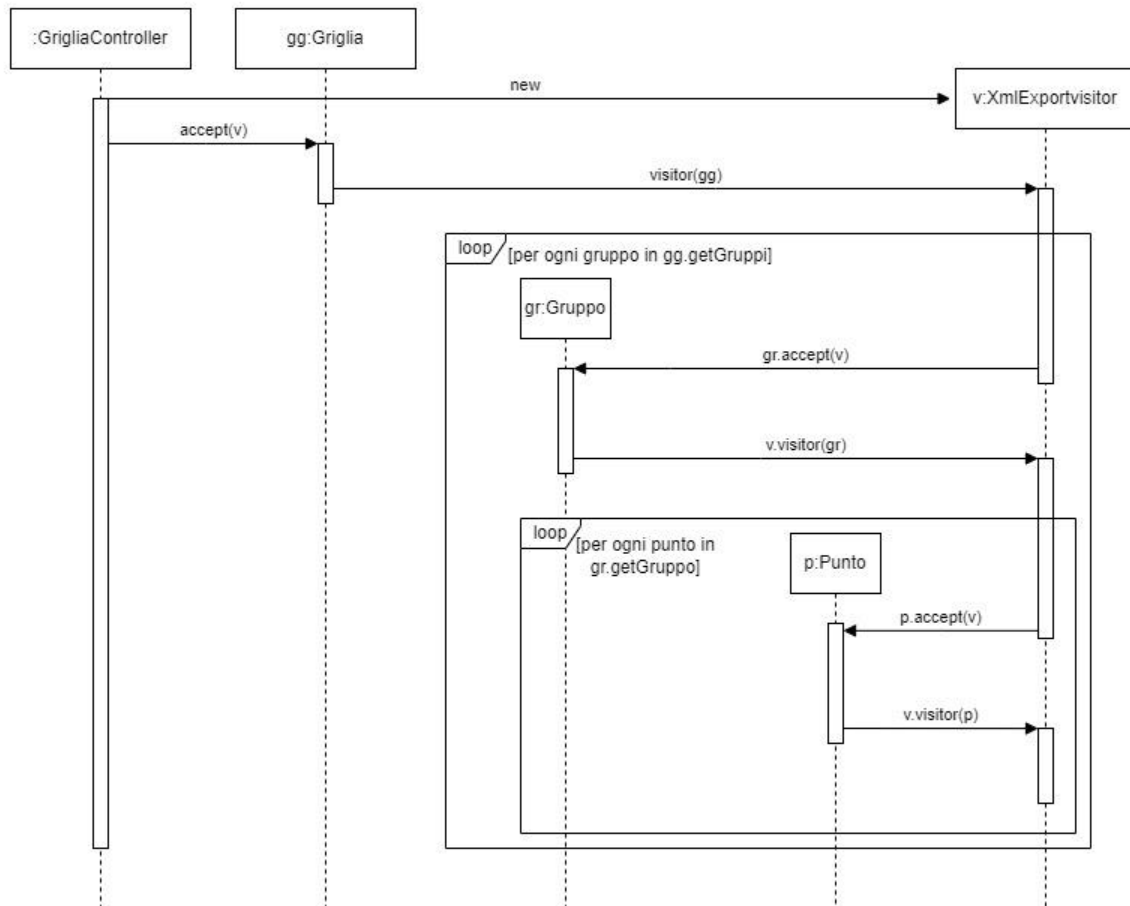


C. Architettura Software

C.1 The static view of the system: Component Diagram

C.2 The dynamic view of the software architecture: Sequence Diagram

Di seguito si illustra un sequence diagram che mostra un possibile flusso di interazione per la visita di un oggetto "Griglia".



D. Dati e loro modellazione (se il sistema si interfaccia con un DBMS)

Non si è riscontrata la necessità di usare un DBMS.

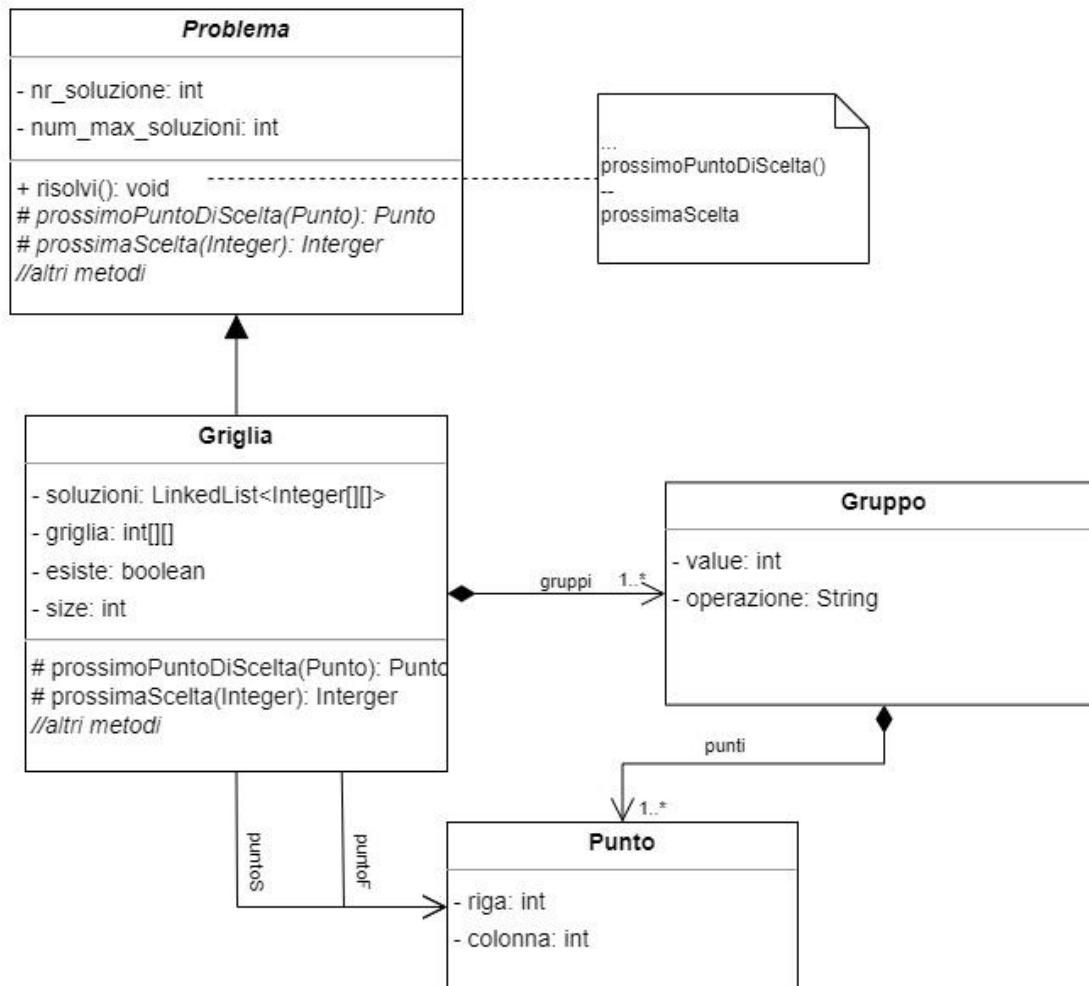
E. Scelte Progettuali (Design Decisions)

Il software è stato realizzato utilizzando i seguenti design pattern:

Utilizzando il pattern **Template Method**, si è potuto implementare un algoritmo ad alto livello di tipo 'backtracking', che, concretizzando i suoi passi, ha consentito di generare una o più soluzioni per una particolare configurazione del gioco KenKen. La struttura di questo algoritmo è stata definita all'interno di una classe base astratta denominata 'Problema', mentre la concretizzazione dei passi specifici dell'algoritmo è stata realizzata tramite una classe concreta denominata 'Griglia'.

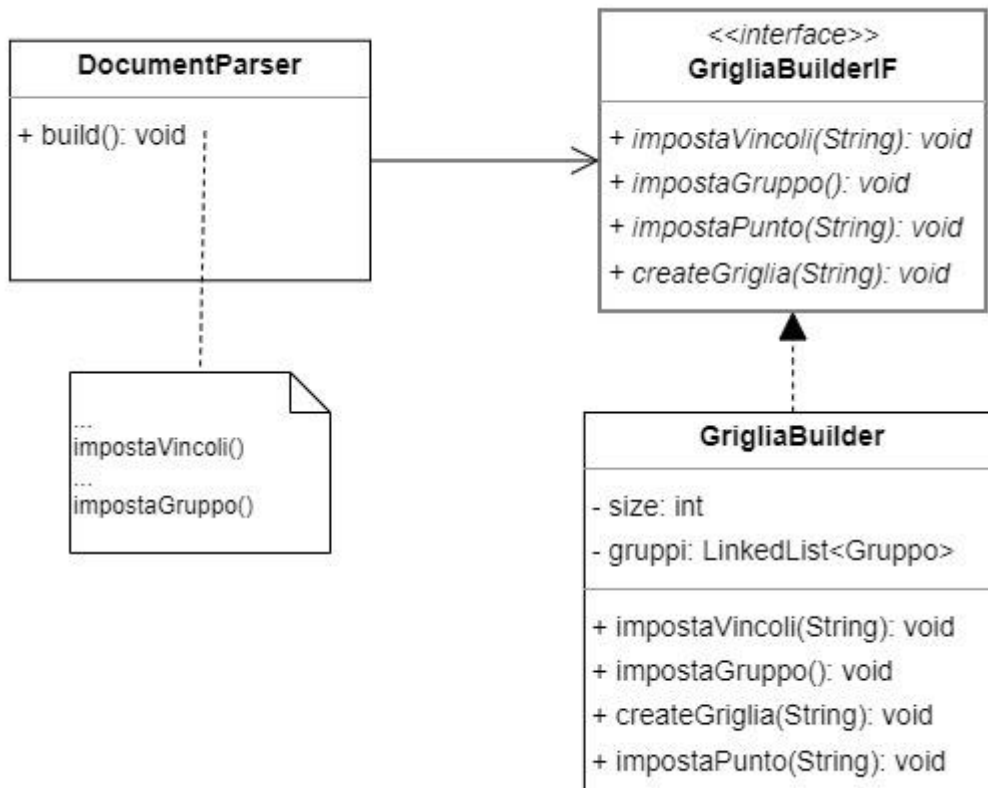
Nel dettaglio, la classe 'Griglia' ha concretizzato i passi dell'algoritmo utilizzando uno stato che rappresenta la configurazione della griglia del gioco KenKen. Questo ha permesso di personalizzare la soluzione dell'algoritmo a seconda della configurazione specifica della griglia e ha consentito di calcolare le soluzioni in modo efficiente e accurato.

Di seguito il corrispettivo class diagram



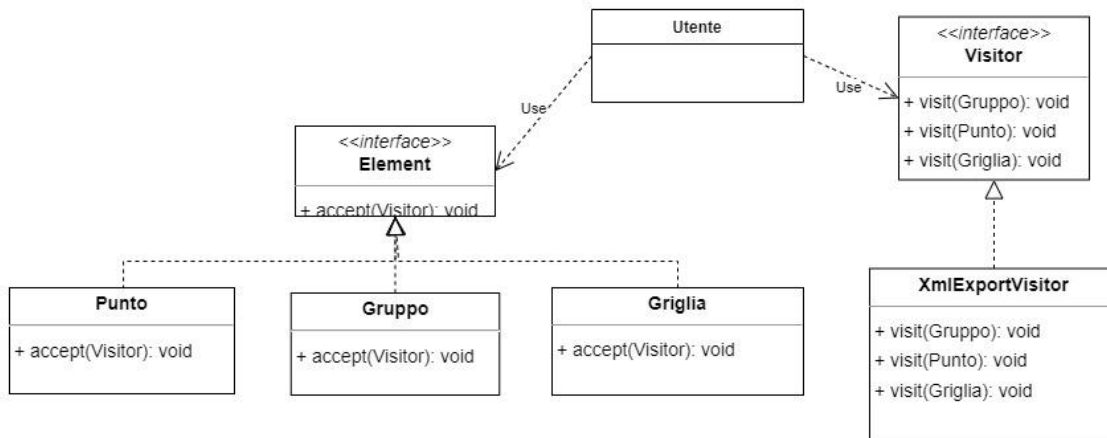
Utilizzando il pattern **Builder**, si è implementata la funzionalità di importazione di una configurazione da un file di tipo .xml. Per gestire questa operazione, si è creata una classe denominata 'DocumentParser', che agisce come direttore, responsabile della lettura del file di input. Successivamente, abbiamo utilizzato un builder concreto chiamato 'GrigliaBuilder' per convertire ogni elemento presente nei tag del file .xml in oggetti Java corrispondenti.

Di seguito il corrispettivo class diagram



Utilizzando il pattern **Visitor**, si è implementata la funzionalità di esportazione di una configurazione di griglia da oggetto java a file di tipo .xml. Per gestire questa operazione, si è creata una classe concreta denominata 'XmlExportVisitor', che implementa tanti metodi quanto sono le classi della struttura. Ogni metodo consente agli oggetti di attraversare la struttura e di rappresentare lo stato dei singoli oggetti sul file .xml. La struttura delle classi che accettano la visita, quindi l'implementazione della funzionalità sono: 'Griglia', 'Gruppo' e 'Punto'.

Di seguito il corrispettivo class diagram



F. Progettazione di Basso Livello

Una volta definite le varie funzionalità del sistema, si sono tradotte in una rappresentazione di più basso livello tramite classi e relazioni tra di esse. Il linguaggio scelto è java.

Per una maggiore organizzazione del codice si è deciso di sfruttare la organizzazione delle classi mediante l'uso di package.

Di seguito vengono riportate le funzionalità di ogni singola classe suddivise per package.

Package **backtracking**:

- L'interfaccia **Element** definisce un contratto. Le classi che la implementeranno dovranno concretizzare il metodo `accept()` che è utilizzato per consentire alla classe di accettare una visita da una classe `Visitor`.
- La classe astratta **Problema** definisce al suo interno il metodo concreto "risolvi()" che definisce l'algoritmo del backtracking. Inoltre, troviamo una serie di metodi astratti che vengono utilizzati dal metodo `risolvi`. Fornisce quindi una base per la definizione di 'problemi' concreti che fanno uso del metodo `risolvi`.
- La classe concreta **Griglia** è utile per rappresentare in memoria una configurazione di gioco. Estende `Problema` e implementa `Element`, quindi concretizza i metodi di `Problema`, nel caso specifico del `kenken`, e provvede a concretizzare il metodo `accept()` utilizzato dal pattern "visitor".
Per poter incapsulare e rappresentare al suo interno una configurazione di gioco, possiede delle dipendenze con oggetti di tipo `Punto` e di tipo `Gruppo`. Offre dei metodi `getter` e `setter` implementando in essi il corretto incapsulamento.
Si noti la presenza del metodo `esisteSoluzione()`, invocato dalla classe `GrigliaController`, che ha la funzione di verificare se la griglia di gioco, inserita dall'utente, possiede almeno una soluzione.
- La classe concreta **Gruppo** implementa `Element` e incapsula ed esprimere in java il concetto di gruppo. In merito a ciò possiede una composizione di almeno un oggetto `Punto` e possiede due variabili (`intera` e `stringa`) per salvare il valore e l'operazione del vincolo. Concretizza il metodo `accept()`.
- La classe concreta **Punto** implementa `Element` e incapsula ed esprime in java il concetto di punto. In merito a ciò utilizza uno stato formato da due variabili

intere per salvare la riga e la colonna. Oltre a dei metodi getter e setter offre metodi di utilità per incrementare e decrementare di una unita riga e colonna. Concretizza il metodo accept().

Package **builder**:

- L'interfaccia **GrigliaBuilderIF** definisce un contratto. Le classi che la implementeranno dovranno concretizzare i metodi in base all'oggetto che vorranno costruire
- La classe concreta **GrigliaBuilder** implementa l'interfaccia GrigliaBuilderIF. Questa classe, concretizzando i metodi dell'interfaccia, è responsabile della costruzione di oggetti di tipo 'Gruppo' e di oggetti di tipo 'Punto per ogni gruppo. I metodi getter sono molto importanti per ritornare gli oggetti creati e quindi utili a istanziare un oggetto 'Griglia'. Inoltre, così facendo evitiamo che un direttore si leghi a una specifica rappresentazione.

Package **director**:

- La classe concreta **DocumentParser** svolge il ruolo di un parser per un documento strutturato in formato .xml, contenente informazioni di una griglia con i rispettivi gruppi e punti. La responsabilità del parser è analizzare il documento e, mediante un oggetto GrigliaBuilderIF concreto che gli viene passato dal client, permette la costruzione di un oggetto Griglia. I metodi implementati all'interno della classe, sono utili per analizzare il documento. Il parser legge il documento riga per riga e suddivide ciascuna riga in parole. Per ogni tag XML che incontra, fornisce una semantica e, di conseguenza, invoca il corretto metodo del builder passandogli i parametri appropriati.

Package **visitor**:

- L'interfaccia **Visitor** definisce un contratto. Le classi che la implementeranno dovranno concretizzare i tre metodi visit() che accettano tre tipi diversi di oggetti come parametri: Griglia, Gruppo e Punto.
- La classe concreta **XmlExportVisitor** implementa l'interfaccia Visitor. Ha la responsabilità di esportare una configurazione di gioco, oggetto di tipo 'Griglia', in un file di formato .xml. L'uso di questa classe consente di visitare i 3 oggetti, Griglia, Gruppo e Punto. Ogni metodo concreto visit(), definisce la sintassi .xml che deve essere usata per convertire lo stato dell'oggetto.

Package progetto.progettokenken:

- La classe concreta **Buttonc** è una sottoclasse di Button, ovvero estende la classe Button di JavaFX. Essa, quindi, permette di personalizzare la classe Button aggiungendo due proprietà e i rispettivi auto esplicativi metodi getter. Le proprietà introdotte rappresentano, l'indice di riga (int riga) e l'indice di colonna (int colonna), quindi, la posizione che un oggetto Buttonc assume all'interno di una matrice di Button utilizzata per far configurare all'utente i gruppi del kenken.
- La classe concreta **TextFieldC** è una sottoclasse di TextField, ovvero estende la classe TextField di JavaFX. Essa, ha la stessa utilità della classe precedente ma, permette di salvare la posizione assunta in una matrice di TextField. Questa matrice è utilizzata dall'utente per poter scrivere i valori. La posizione serve per poter capire che posizione assume all'interno matrice altrimenti non si riuscirebbe a determinare la correttezza dei valori rispetto ai vincoli.

La parte grafica è stata implementata utilizzando il framework JavaFX. Ha permesso la realizzazione di un'interfaccia utente intuitiva e user-friendly. L'interazione tra gli elementi grafici e le varie funzionalità del software è stata gestita in maniera semplice utilizzando l'annotazione @FXML (di javaFX) al di sopra dei metodi responsabili della gestione degli eventi generati dall'interazione con i componenti grafici.

- La classe concreta **GrigliaController** è un controller per l'interfaccia grafica definita nel file "kenken-view.fxml". L'interfaccia definita nel foglio .fxml possiede un insieme di oggetti grafici quali: Button, TextField, MenuItem ecc.

Essendo un controller per la GUI, all'interno si trovano numerosi metodi. I metodi su cui al di sopra è riportata l'annotazione @FXML sono quelli a cui è associata un evento dell'oggetto grafico. In questo software l'evento a cui si deve rispondere è il solo click con il mouse. I restanti metodi sono di utilità.

G. Spiegare come il progetto soddisfa i requisiti funzionali (FRs) e quelli non funzionali (NFRs)

Requisiti funzionali

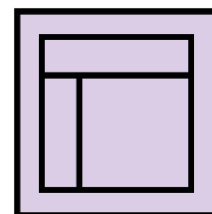
1. *Creazione dei puzzle*, reso possibile grazie all'impiego del framework grafico javaFX e al corretto impiego delle classi che incapsulano l'istanza di gioco.
2. *Importazione/esportazione della griglia su da un file*: reso possibile grazie al corretto impiego dei pattern Visitor e Builder.
3. *Supporto per diverse dimensioni di puzzle*: reso possibile grazie all'impiego del framework grafico javaFX. Si è fatto uso della dinamicità nella creazione delle griglie di gioco.
4. *Verifica della soluzione dell'utente*: reso possibile grazie al corretto impiego del pattern TemplateMethod. Fornita una soluzione ad una griglia di gioco (identificata da una matrice), si procede a iterare sugli elementi numerici della matrice e a verificare che ognuno di essi è assegnabile tramite il metodo "assegnabile()" della classe Griglia.
5. *Funzionalità di annullamento dei gruppi in fase di configurazione*: reso possibile grazie all'impiego del framework grafico javaFX tramite dei button che permettono di annullare la precedente mossa.
6. *Visualizzazione e iterazione delle soluzioni*: reso possibile grazie al corretto impiego dei pattern TemplateMethod. Si invoca il metodo risolvi(), ogni singola soluzione viene salvata in una lista. Le matrici di interi vengono mostrate successivamente sulla griglia di JTextField.
Tramite due pulsanti è possibile iterare sulla lista mostrando le varie soluzioni.

Requisiti non funzionali

1. **Usabilità**, il software del kenken è stata progettato per essere intuitivo e facile da usare. L'interfaccia utente offre un utilizzo semplice, esprime in modo chiaro le varie funzionalità, inoltre, la presenza di riscontri permette all'utente di conoscere eventuali errori e di essere guidato nelle azioni da intraprendere. Tutto ciò grazie all'uso di javaFX.
2. **Prestazioni**, per quanto il software possa essere stato ottimizzato nelle sue vari parti, la presenza dell'algoritmo di backtraking con una complessità $O(n^2)$, su

- determinati input (quali griglia superiore a 6x6, numeri elevati da soluzioni da dover fornire) può rallentare significativamente il sistema.
3. **Manutenibilità**, l'impiegato dei design pattern ha garantito ordine nella stesura del codice e maggiore facilità nella comprensione dello stesso. Inoltre, l'utilizzo di commenti ha reso il codice maggiormente comprensibile. Esempio di evolvibilità: tramite il pattern builder è possibile definire una classe che dato un file .xml lo converte in altri formati.
 4. **Affidabilità**, il gioco deve essere stabile e affidabile, evitando crash o comportamenti imprevisti. Richiede un'attenta gestione degli errori, test.
 5. **Robustezza**, l'impiego di controlli vari nel codice, e la stampa a video di eventuali errori, permette al software di fornire comportamenti accettabili anche in circostanze non previste. Ciò evita di far terminare l'esecuzione in maniera accidentale.
 6. **Portabilità**, la realizzazione è stata resa possibile grazie all'impiego del linguaggio di programmazione Java. Il suo bytecode generato viene eseguito dalla Java Virtual Machine su diverse piattaforme hardware e software. In questo modo si astrae dallo specifico sistema operativo o architettura potendo evitare di dover effettuare modifiche al codice sorgente.

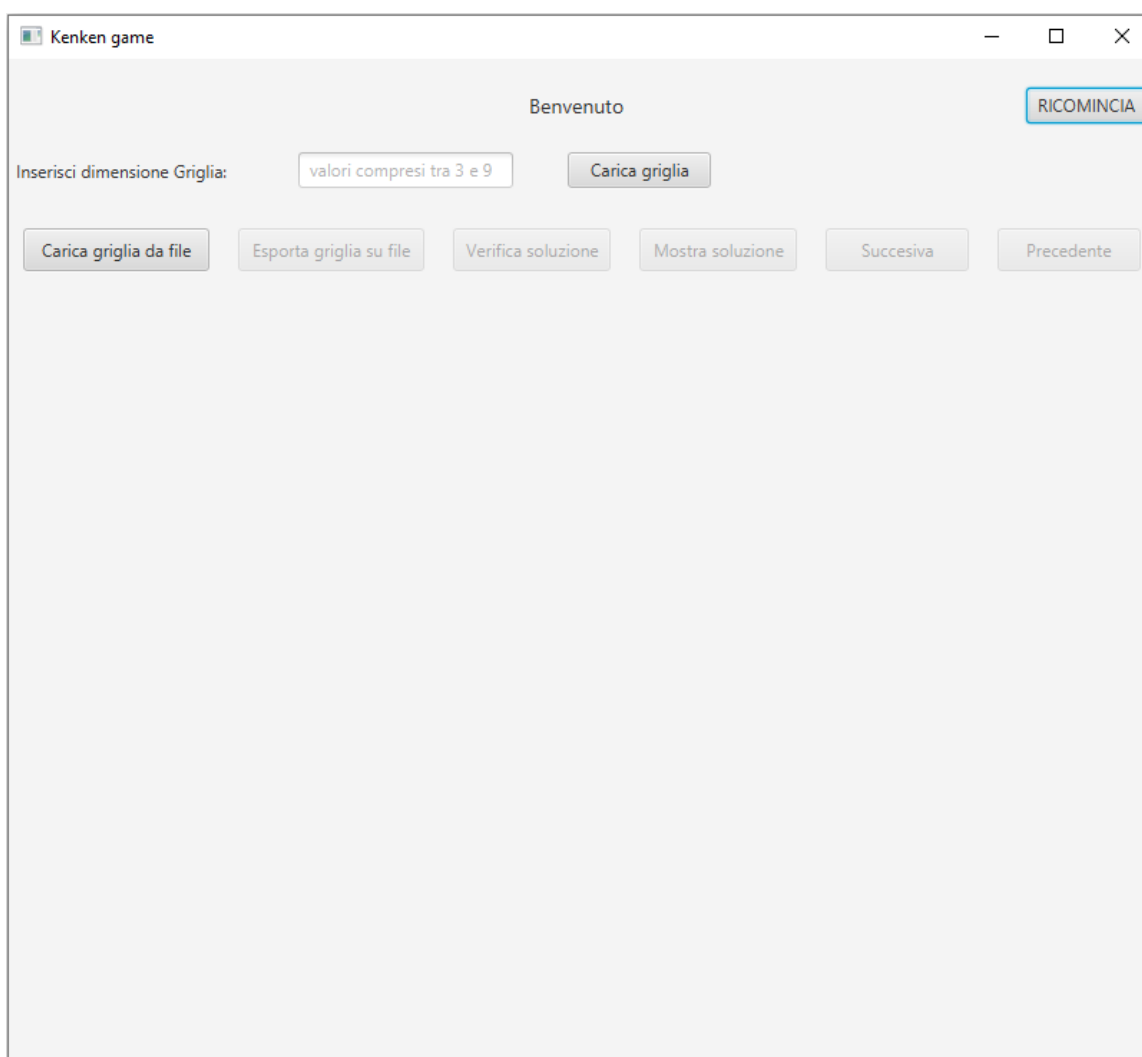
Appendix. Prototype



Di seguito si fornisce una breve descrizione del prototipo realizzato riguardo il gioco del KenKen.

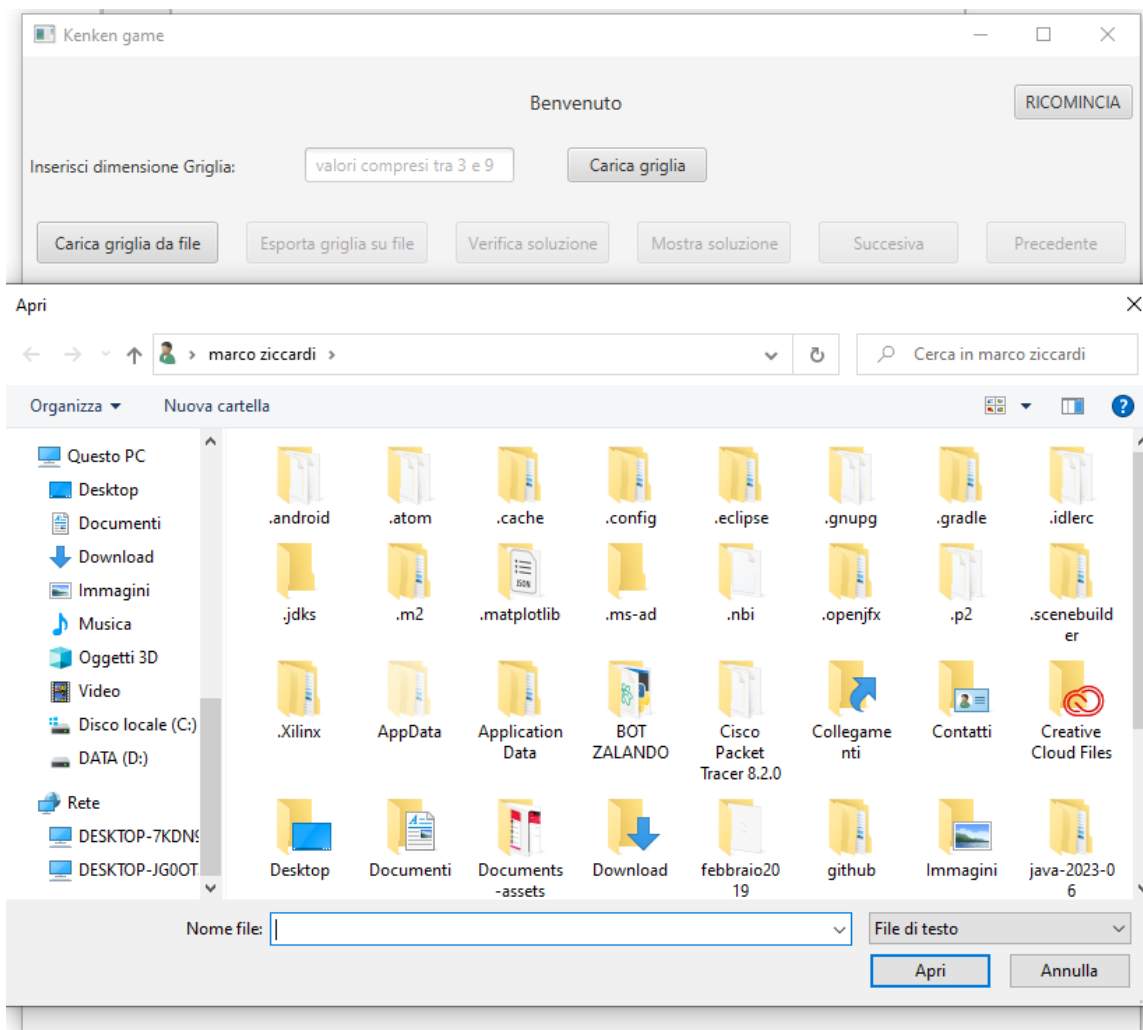
Caricamento configurazione di gioco

All'avvio, l'applicazione si presenta come riportato nella foto di seguito.

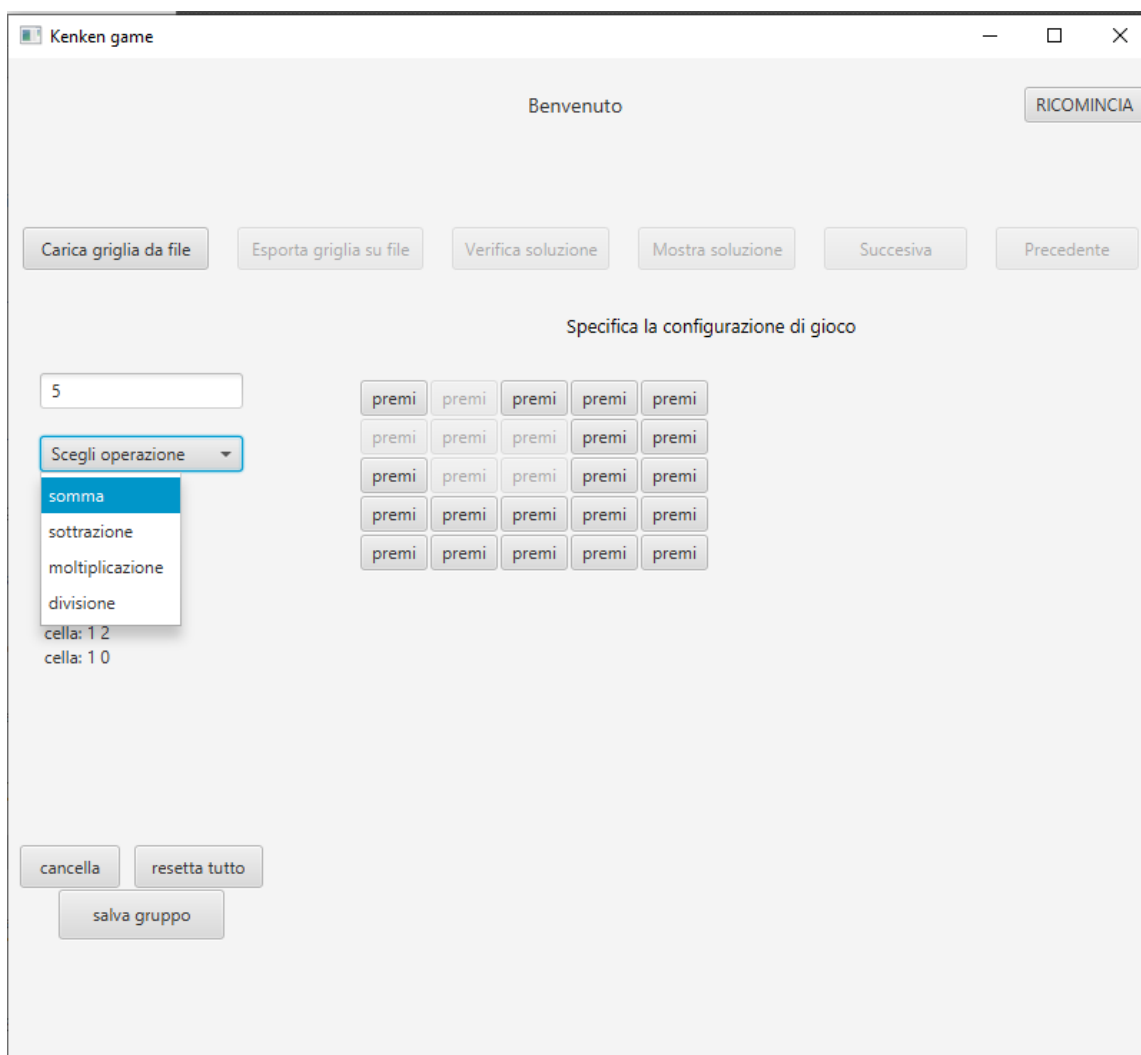


Come da requisito, all'inizio, viene data la possibilità all'utente di poter inserire una griglia da file o manualmente.

L'inserimento da file è reso possibile dalla apertura di un File chooser, in seguito al click del pulsante "Carica griglia da file", che permette di inserire un file

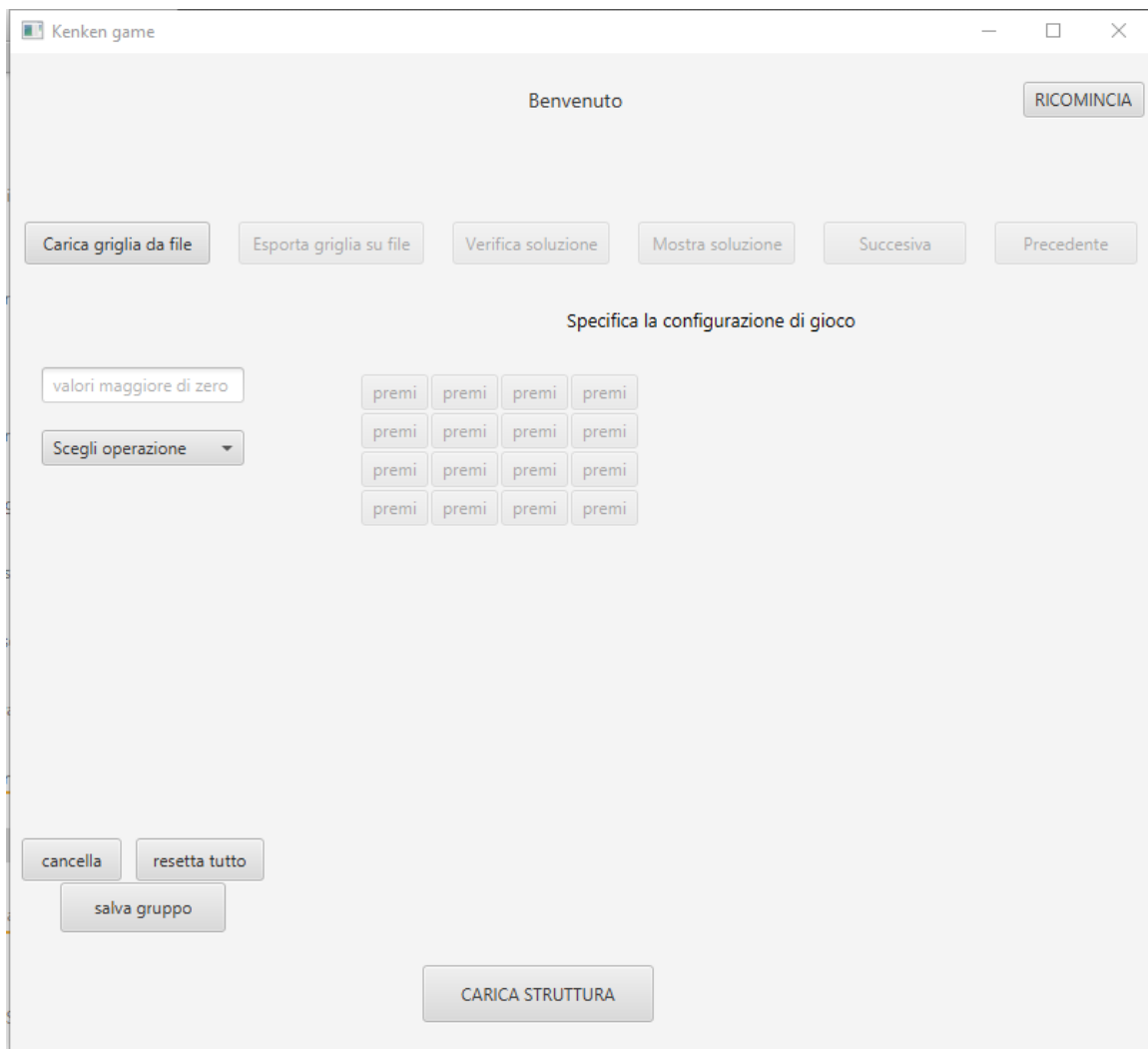


Per accedere alla configurazione manuale, l'utente deve inizialmente specificare la dimensione desiderata nella apposita TextField e, di seguito al click del mouse sul pulsante "Carica Griglia" si apre la schermata di configurazione. Di seguito lo screen corrispondente.



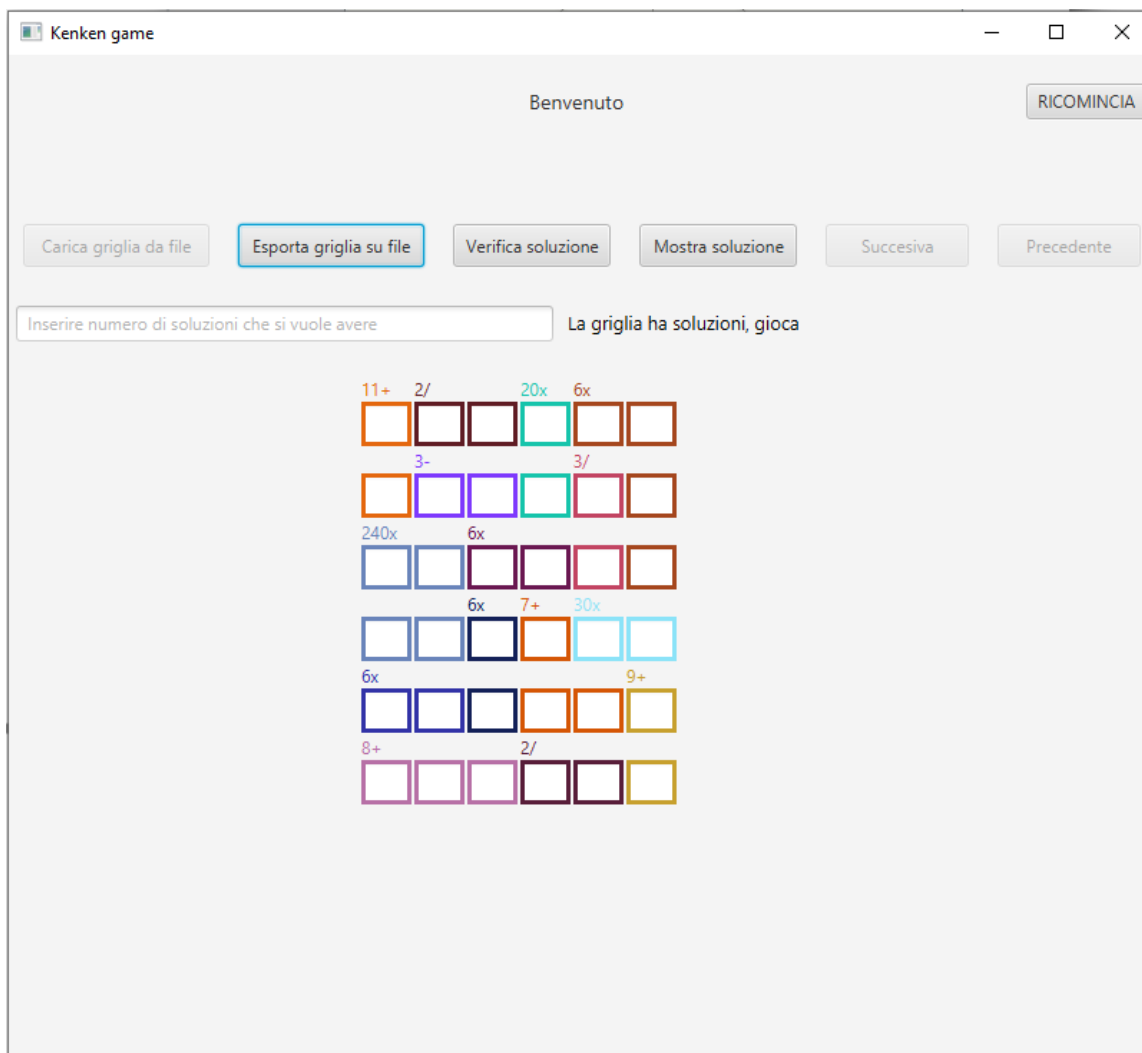
(foto 3)

Si noti come, nella foto 3, viene data la possibilità all'utente di specificare i vari gruppi. I Button che risultano essere premuti rappresentano la "forma" dell'i-esimo gruppo. A sinistra è data la possibilità di specificare i vincoli matematici come valore e operazione. Il Button "cancella" è utile per annullare la selezione corrente, il Button "resetta tutto" è utile per annullare tutte le selezioni di gruppi effettuate e infine il Button "salva gruppo" serve per poter salvare l'i-esimo gruppo e poter passare alla selezione del successivo.



Selezionati tutti i pulsanti (equivalente a dire: si sono specificati tutti i gruppi) l'utente può procedere al caricamento della griglia di gioco cliccando il button "carica struttura".

Caricamento configurazione di gioco “avente almeno una soluzione”



Questa schermata permette all’utente di poter giocare. Su ogni singola celletta e possibile inserire un valore numerico.

Le funzionalità permesse in questa schermata sono:

- poter esportare la configurazione di gioco su un file tramite il buttom “Esporta griglia su file” il quale aprire un file chooser;
- il click sul buttom “Verifica soluzione” permette di verificare se i valori scritti siano corretti o presentano errore, inoltre se i valori ricoprono l’intera griglia permette di verificare se il gioco è risolto.

Di seguito sono riportati due schermate che fanno vedere come si presenta il controllo dei valori nei due casi: presenza di errori, correttezza dei valori.

Verifica correttezza dei valori (valori corretti)

Kenken game

Benvenuto RICOMINCIA

Carica griglia da file Esporta griglia su file Verifica soluzione Mostra soluzione Succesiva Precedente

Inserire numero di soluzioni che si vuole avere Valori corretti

11+	2/		20x	6x	
1	2				
	3-			3/	
240x		6x			
		6x	7+	30x	
6x					9+
8+		2/			

Verifica correttezza dei valori (valori errati)

Kenken game

Benvenuto

RICOMINCIA

Carica griglia da file Esporta griglia su file Verifica soluzione Mostra soluzione Successiva Precedente

Inserire numero di soluzioni che si vuole avere

Non tutti i valori inseriti rispettano i vincoli. Reinserire

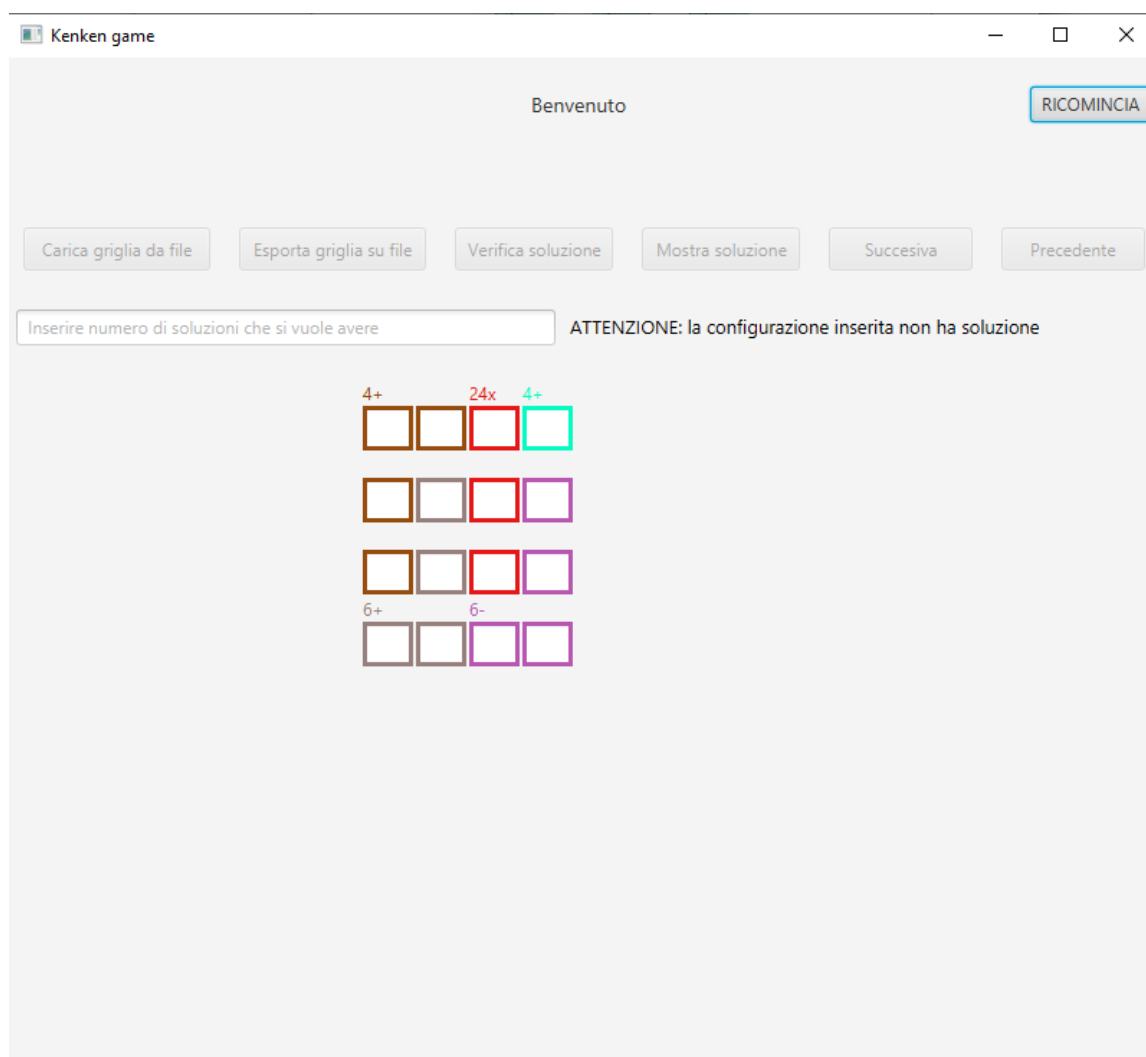
11+	2/		20x	6x	
1	2	2			
	3-		3/		
240x	6x				
	6x	7+	30x		
6x				9+	
8+	2/				

Caricamento configurazione di gioco “non avente nessun soluzione”

Per ovvi motivi, la configurazione di gioco scelta dall’utente può non avere soluzioni.

Nel caso in cui la configurazione di gioco non possiede almeno una soluzione, viene mostrata a video che non possiede soluzione. Non è attiva nessuna funzionalità precedentemente descritta.

Di seguito la schermata di come si presenta.

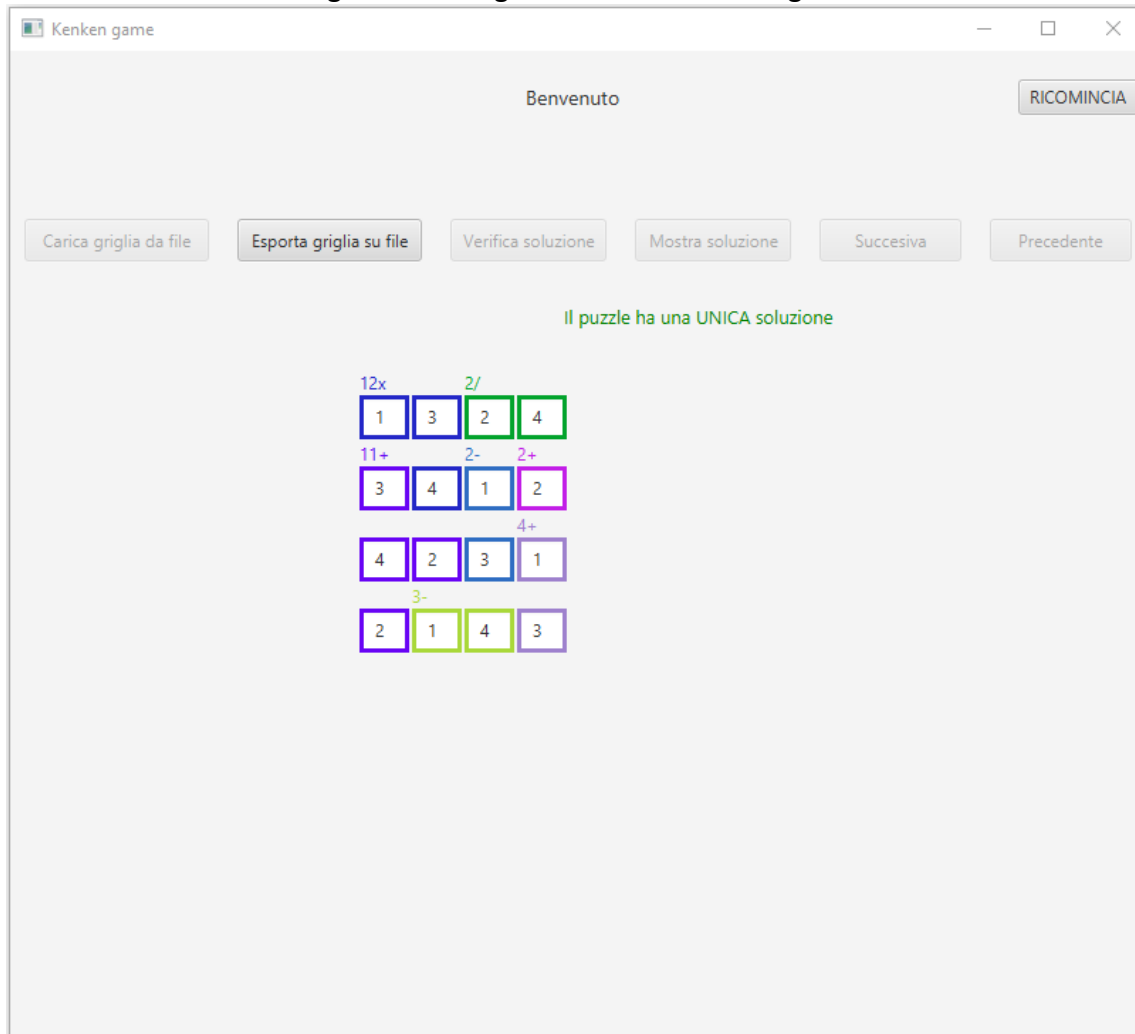


Mostra risoluzione by software

L'utente può esprimere la volontà di farsi mostrare le possibili 'n' soluzioni dal software. Ciò è reso possibile dal button "Mostra soluzione" il quale, solo dopo aver inserito il numero di soluzioni desiderate le mostra sulla griglia di gioco.

Nel caso in cui vi sono più soluzioni, si attivano i pulsanti "Successiva" e "Precedente" che permettono di iterare sullo spazio di soluzioni generato.

Schermata con una configurazione di gioco che ammette singola soluzione.



Schermata con una configurazione di gioco che ammette soluzioni multiple. Si notino i pulsanti “Successiva” e “Precedente” attivi.

